

# dogfooding试标文档

## 📖 前置说明

### 众测平台：

- 1. Trea CN，需要众测人员自行在 [国内版 Trea 官网](#) 进行下载安装。
- 2. 需要进入ppe泳道，进入方式见 [泳道开启方式](#)。

### 题目采集来源：

- 1. 符合语言类型的 GitHub 公开项目。
- 2. 众测人员过去自建的符合语言类型项目。

### 题目采集要求：

- 1. 符合指定的语言类型。
- 2. 在 GitHub 上有公开的 Repo 和 PR（可自建）。
- 3. 采集时也需要给出对应 Repo 及 PR 的链接。

---

## 🌐 众测 SOP

### 题目收集

根据指定的语言类型，在 GitHub 收集相应的题目（可自建），需要记录：

- 1. Prompt
  - 基于 Repo 及其它 PR 构建的 Prompt，可以复用其他 PR 的内容，不过需要前置说明 PR 前已实现的功能。
- 2. Repo 链接
- 3. Repo 介绍（简介）
- 4. Repo 使用的技术栈

需要大家尽可能地探索模型的知识面或是其他方面能力的欠缺，并具体写出具体方向/类型，不要太过笼统。

### 众测流程（每个模型都需要单独测试）

- 本期需要测试的模型为：kimi k2.5，glm5.0
- 本期测试题量：200

#### 1. 基于分发的题目在 Trea 上进行测试

- 注意：
  - i. 每个题目都需要新起一个对话
  - ii. 尽可能模拟真实用户场景，遇到不合理的可以打断，尝试重新人工引导修复1-2次。
  - iii. 如果模型表现严重不合理的，可以备注对本次答题非常失望，拒绝尝试人工引导修复。

#### 2. 将模型输出结果 PR 到 Repo 的中

#### 3. 基于测试结果在表格中填写：

- a. Session id

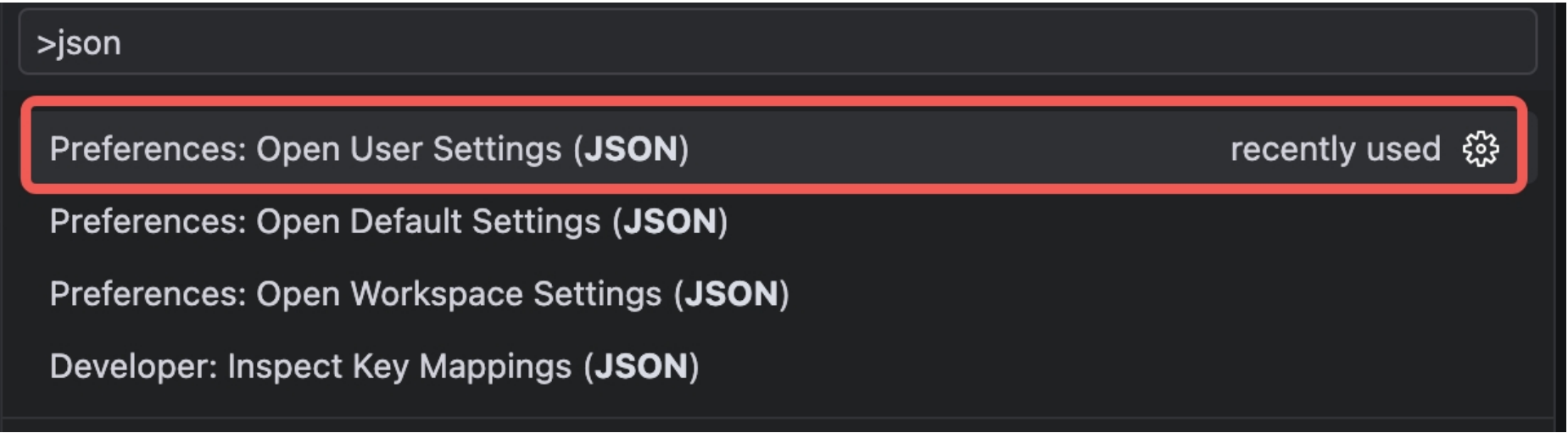
- b. PR 链接
- c. 模型表现打分（详情见附录）
- d. 如果用户体验满意度不为 5 分，则需要填写【问题类型】、【问题描述】、【修复成本】（详情见附录）
- e. 如果用户体验满意度为 5 分，则需要填写模型优点，站在用户角度尽可能详细说明模型做的好的地方。
- f. 在所有被测模型单独打分完成以后，对模型进行两两对比，打GSB标签。
  - GSB 标签可以基于前置的打分，也可以基于大家的主观判断，说明合理即可。
  - 如果主观觉得两个模型间存在差距，需要说明好的模型好在哪，坏的模型坏在哪，简单说明即可。

## Appendix

### 泳道开启方式

1. 打开用户json设置

F1键或点击搜索，输出>json，点击选择下图所示内容：



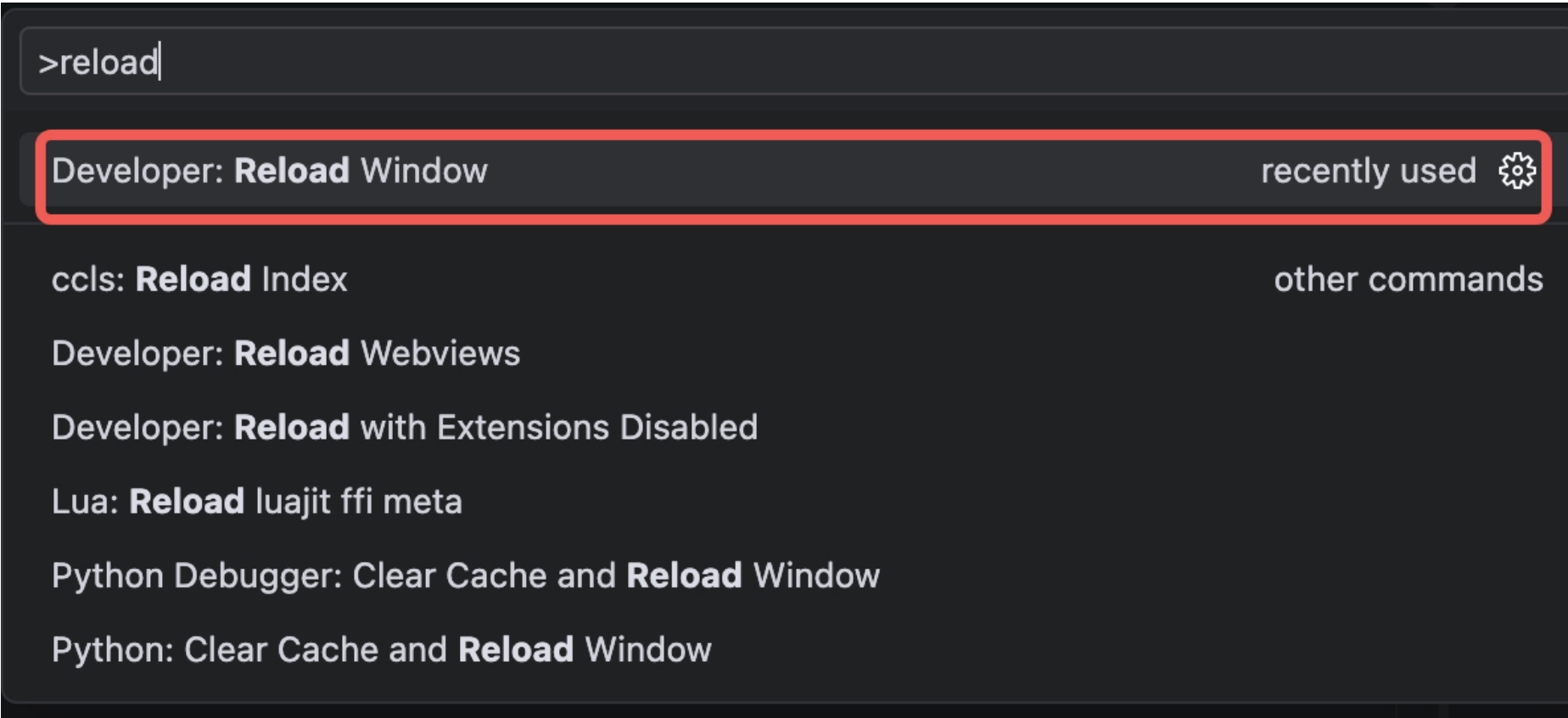
2. 添加ppe配置:

在打开的 `settings.json` 中，添加以下两行并保存：

代码块

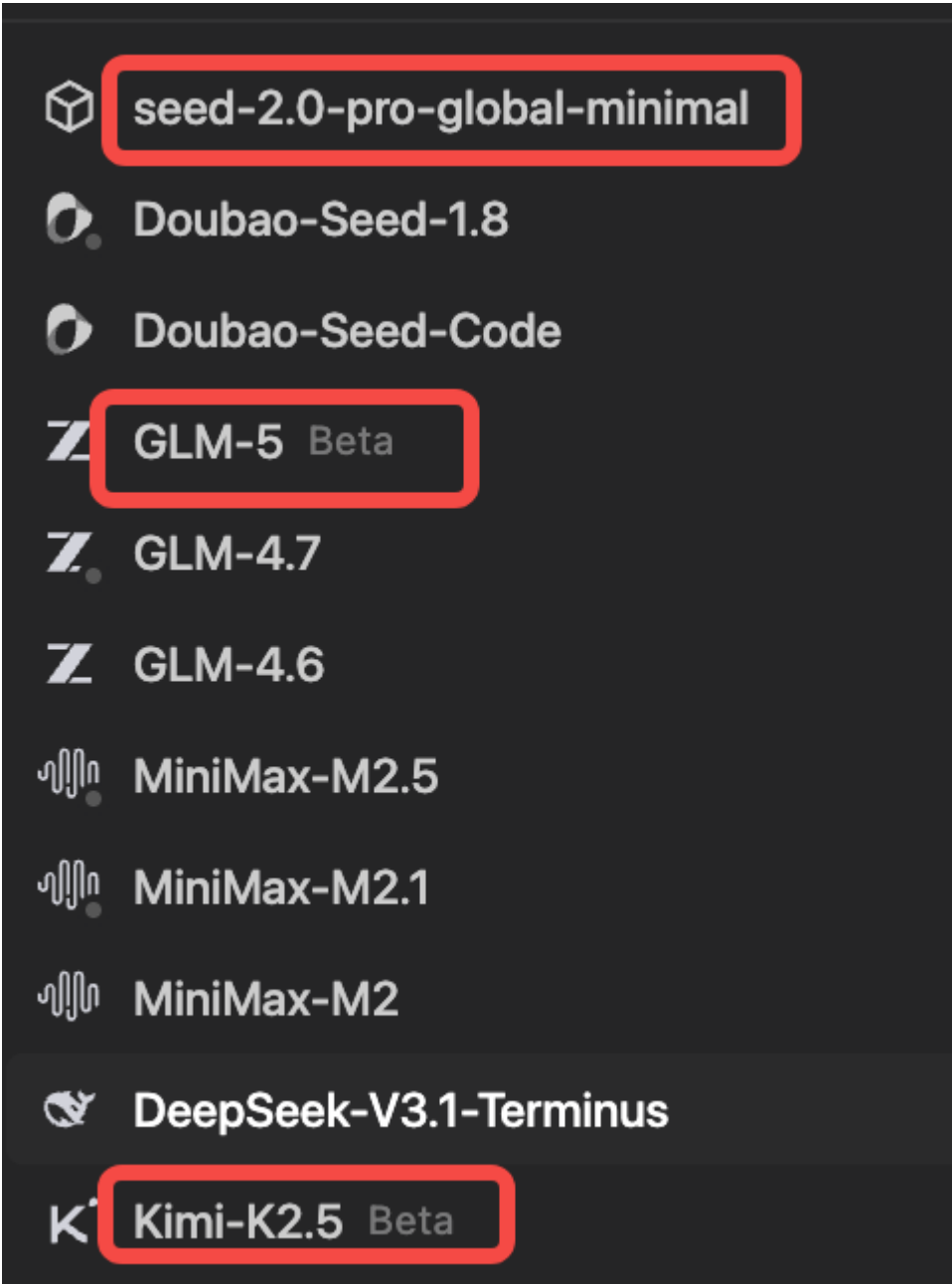
```
1      "ai_assistant.request.env": "ppe",
2      "ai_assistant.request.ppe": "ppe_trae_seed_code_internal_4"
```

3. reload window



3. 选择内置模型（会变化）

再次进入模型选择界面即可看见我们的待测模型，`seed-2.0-pro-global-minimal`，`GLM-5` 以及 `Kimi-K2.5`。



模型表现打分详情：

| 打分维度     | 5分   | 4分   | 3分  | 2分  | 1分    |
|----------|------|------|-----|-----|-------|
| 用户体验满意度： | 非常满意 | 大致满意 | 一般般 | 不满意 | 非常不满意 |

|  |                                      |                                                                                                                                                                                                                                                         |                                                                                                                                                                                                          |                                                                                                                                                                                     |                                                                                                                                                                           |                                                                                                                                        |
|--|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
|  | 站在用户角度，对模型整体表现进行打分，可以带主观意见，打分说明仅供参考。 | 模型推理高效流畅、结构清晰、几乎无停顿或自纠环节，且能迅速抓住问题核心。输出结果让人愉快且可信，几乎无改进空间。                                                                                                                                                                                                | 模型整体体验良好，偶有轻微冗余操作自我修正，但不影响对任务的完成。                                                                                                                                                                        | 模型存在一定的重复或冗余解释。如：存在较多的自纠环节，偶尔将简单问题复杂化；用户仍有意愿等待模型结果                                                                                                                                  | 模型推理过程拖沓、反复纠正或出现明显逻辑错误。如：简单问题被过度复杂化。导致用户失去耐心，需要中断模型推理并进程进行干预。                                                                                                             | 模型推理步骤极其繁琐或陷入死循环且无自我修复能力。严重影响用户体验且没有任何有效阶段性产出。                                                                                         |
|  | 规划&执行反馈                              | <ul style="list-style-type: none"><li>有详细且合理的规划，大任务和子任务拆分合理</li><li>在复杂任务下主动创建 todo/checklist 或等价任务清单</li><li>todo 状态会随着执行实时更新，完成项、进行中项、阻塞项清晰</li><li>工具使用合理（todo、checklist、阶段设计等）</li><li>阶段性总结和反馈清晰且详细</li><li>遇到歧义时能主动确认</li><li>状态追踪合理且稳定</li></ul> | <ul style="list-style-type: none"><li>有详细且合理的规划，大任务和子任务拆分合理</li><li>有合理的 todo/checklist，但更新不够及时，或个别子任务状态与实际执行不完全一致。</li><li>25%工具使用略微不合理或缺失</li><li>阶段性总结和反馈清晰</li><li>追踪合理，实时在工具中更新大任务和子任务的状态</li></ul> | <ul style="list-style-type: none"><li>有合理的大任务规划，但 todo/checklist 不完整，或只在开头列出，后续缺少更新，导致执行可追踪性一般。</li><li>50% 部分工具使用不合理或缺失有阶段性总结和反馈，但不够清晰和明确</li><li>有状态追踪，但不够详细，合理，或没用工具更新</li></ul> | <ul style="list-style-type: none"><li>有模糊大任务规划，但不合理或不明确。缺少有效 todo/checklist；执行过程中状态追踪断续，容易遗漏子任务。</li><li>大部分工具使用都不合理或缺失阶段性总结和反馈时有时无</li><li>有状态追踪时有时无，或小部分错误/延时</li></ul> | <ul style="list-style-type: none"><li>无合理的大任务规划，无子任务拆分，无 todo/checklist、无状态更新。</li><li>无工具使用</li><li>无阶段性总结和反馈</li><li>无状态追踪</li></ul> |
|  | 理解/推理能力：<br><br>代码+自然语言(指令遵循)        |                                                                                                                                                                                                                                                         |                                                                                                                                                                                                          |                                                                                                                                                                                     |                                                                                                                                                                           |                                                                                                                                        |

|                                     |                                                                                                                                                                              |                                                                                                                                                                            |                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                         |                                                                                                                                                                                                         |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                     | <ul style="list-style-type: none"><li>能够准确、精炼地整合全部上下文并整合 codebase，保留所有关键信息并得出正确结论，同时遵循指令且思路连贯。</li><li>做出最少最精炼的代码更改</li><li>无需人工引导</li><li>模型有宽且广的知识面去理解任务并高效完成任务。</li></ul> | <ul style="list-style-type: none"><li>能够整合上下文内容，遵循指令并不遗漏关键信息，给出正确的结论。思维链偶有冗余。</li><li>在正确的位置做正确的代码更改。</li><li>无需人工引导。</li><li>模型有足够的知识面去理解并完成任务，不一定高效，或者优化做的不够好。</li></ul> | <ul style="list-style-type: none"><li>能够整合指令中大部分上下文内容，但在信息传递过程中丢失细节或约束，导致交付成果存在缺陷。</li><li>可能存在轻微指令偏离或规划不当。</li><li>能够理解并整合 codebase，能保留大部分有关键信息并在正确的位置做正确的代码更改，但有所丢失，或者忽略代码中的约束和细节，存在轻微偏离。</li><li>人工引导后可以正确完成任务。</li><li>模型有勉强的知识面去去理解并完成任务里的主线任务，但是支线或者次要任务没办法完成。</li></ul> | <ul style="list-style-type: none"><li>误解部分指令中上下文内容，致使模型遗漏关键信息，造成思维存在重大缺失。</li><li>人工引导后能完成部分任务。</li><li>误解代码框架中的大部分内容，不知道应该更改什么文件，或者错误更改不应该被修改的代码内容。</li><li>模型仅有少量相关知识，能理解小部分内容，但是容易偏题，无法交付。</li></ul> | <ul style="list-style-type: none"><li>完全忽略或错误解读上下文，致使模型思维中断或违反硬性约束条件的步骤。</li><li>完全误解 codebase，无法理解代码框架，不知道应该更改什么文件，或者错误更改不应该被修改的代码内容人工引导后也无法完成任务中的任何部分。</li><li>模型几乎没有相关知识，无法理解意图，无法交付，盲目自信。</li></ul> |
| <b>复杂指令遵循：</b><br><br>多约束条件的持续遵循能力。 | 完美遵循所有多重约束条件，并在后续修复、补充实现、调试过程中始终不破坏前置要求；不会因任务                                                                                                                                | 初次输出遗漏了 1-2 个 <b>非核心</b> 的约束（如某个变量命名未按要求），经用户 <b>指出 1 次后</b> ，能立刻全部纠正并交付。                                                                                                  | 初次输出忽略了关键约束（如改了不该改的文件），经过 <b>2 次明确警告和纠正后</b> ，勉强能将代码调整到                                                                                                                                                                                                                        | 经过 <b>2 次纠正后</b> ，模型依然无法同时满足多个约束（顾此失彼，修了这个忘了那个），导致最终产出不符合要求。                                                                                                                                            | 彻底无视复杂的约束条件，只顾按自己的默认习惯输出，甚至在人工纠正时陷入死循环或狡辩，严重破坏原有结构。                                                                                                                                                     |

|                                                                                                      |                                                                        |                                                |                                    |                                                |                                         |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|------------------------------------------------|------------------------------------|------------------------------------------------|-----------------------------------------|
|                                                                                                      | 变复杂而遗忘限制或擅自扩大修改范围。                                                     |                                                | 符合所有约束的状态。                         |                                                |                                         |
| <b>工程完备度：</b><br><br>重点评估模型在实现功能或修复问题时，是否具备基本工程质量意识，包括是否主动补充/更新测试、是否复用现有测试框架、是否进行必要验证、是否避免只交付“表面代码”。 | 能根据改动类型主动补充或更新测试；测试位置正确、覆盖核心路径，与现有测试风格一致；若任务明显需要验证，模型会明确说明验证方式或补齐关键测试。 | 大多数情况下能补测试或指出应补测试；测试基本有效，但覆盖面略窄，或与现有工程风格有轻微偏差。 | 在用户提醒后才补测试；测试只覆盖部分主路径，或只补了最基础case。 | 多次提醒后才尝试写测试，但测试位置、框架、断言或覆盖范围存在明显问题，难以真正保障改动质量。 | 完全没有测试意识；即使任务明显需要测试也不写，或写出不可运行/无意义的伪测试。 |
|                                                                                                      |                                                                        |                                                |                                    |                                                |                                         |
|                                                                                                      |                                                                        |                                                |                                    |                                                |                                         |
|                                                                                                      |                                                                        |                                                |                                    |                                                |                                         |

问题类型

罗列模型存在的问题选择（多选）：

- **幻觉**：引用不存在的库、API、变量或文件路径。
- **上下文丢失**：忽略了 codebase 中已有的工具类，或忘记了上一轮的对话约束。
- **指令遵循失败**：明确说了“不要动配置文件”，但它还是改了；或者没按要求输出格式。
- **死循环**：指出了错误，模型道歉但下一轮输出一模一样的错误代码。
- **偷懒**：省略主要代码或提供伪代码，且无法通过指令让其补全。
- **代码破坏**：修复了一个 bug，引发了新 bug；或破坏了原有的正确逻辑。
- **代码Bug**：生成的代码存在bug，无法运行。
- **废话过多**：代码很少，解释性废话极多，且无法抓住重点。
- **输出中断**：模型调用工具过程中出错或是其他原因导致输出突然中断。
- **目标漂移 / 分心**：执行过程中偏离用户主目标，长时间处理无关问题、边缘优化或重复解释，影响主任务完成。
- **其他**：其他问题，可以是模型风格等主观体感问题，需要在问题描述中说明。

修复成本

作为用户，为了让模型输出正确结果，修复成本有多高？

- **低 (Low)**：
  - 模型犯了小错，**提醒了一次**，它立刻完美修正了。
- **中 (Medium)**：

- 模型理解有偏差， **反复说明了 2 次**，或者需要把报错日志贴给它，它才修好。
- **高 (High) :**
  - **引导超过 2 次** 仍未解决；
  - 或者模型陷入死循环；
  - 或者必须自己动手写代码才能解决问题（模型经过提示说明后不可用）。